In the examples in chapter 8 that explicitly spawned threads, after dividing the work between threads, the master thread always waited for the newly spawned threads to finish, to ensure that the overall task was complete before returning to the caller. With thread pools, you'd need to wait for the *tasks* submitted to the thread pool to complete, rather than the worker threads themselves. This is similar to the way that the `std::async`-based examples in chapter 8 waited for the futures. With the simple thread pool from listing 9.1, you'd have to do this manually using the techniques from chapter 4: condition variables and futures. This adds complexity to the code; it would be better if you could wait for the tasks directly.

By moving that complexity into the thread pool itself, you *can* wait for the tasks directly. You can have the `submit()` function return a task handle of some description that you can then use to wait for the task to complete. This task handle would wrap the use of condition variables or futures, thus simplifying the code that uses the thread pool.

A special case of having to wait for the spawned task to finish occurs when the main thread needs a result computed by the task. You've seen this in examples throughout the book, such as the `parallel_accumulate()` function from chapter 2. In this case, you can combine the waiting with the result transfer through the use of futures. Listing 9.2 shows the changes required to the simple thread pool that allows you to wait for tasks to complete and then pass return values from the task to the waiting thread. Since `std::packaged_task<>` instances are not *copyable*, just *movable*, you can no longer use `std::function<>` for the queue entries, because `std::function<>` requires that the stored function objects are copy-constructible. Instead, you must use a custom function wrapper that can handle move-only types. This is a simple type-erasure class with a function call operator. You only need to handle functions that take no parameters and return `void`, so this is a straightforward virtual call in the implementation.

Listing 9.2. A thread pool with waitable tasks